Question Bank — Week 1

Why this course · Python basics (Lecture 01, Lab 01) — Questions

DSA27 · Data Structures and Algorithms

Fall 2026

Contents

Part A — Multiple choice **1**

Part B — Short answer and essay **8**

Part C — Trace the code **9**

Part D — Find and fix the bug **10**

Part E — Write the code **10**

> ***Answers are in a separate file:*** `week01-answers.md`. *Attempt every question on paper first. Each question is tagged with its level:* ***[what]*** *— recall,* ***[how]*** *— apply,* ***[why]*** *— explain and justify. Marks shown for written questions are a guide to the length of answer expected.*

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W1-M01 – W1-M42
B	Short answer and essay	W1-E1 – W1-E10
C	Trace the code — what is printed?	W1-T1 – W1-T6
D	Find and fix the bug	W1-B1 – W1-B3
E	Write the code — checked by `pytest`	W1-C1 – W1-C4

Part A — Multiple choice

W1-M01 [what] "Every general-purpose programming language is Turing-complete." What does that tell you?

- **a)** All languages run equally fast
- **b)** Any problem solvable in one of them is solvable in any other
- **c)** All languages differ only in syntax
- **d)** A program written in one language runs unchanged in any other

W1-M02 [what] Python's type system is best described as:

- **a)** static and strong
- **b)** static and weak
- **c)** dynamic and weak
- **d)** dynamic and strong

W1-M03 [how] What does `"3" + 5` do in Python?

- **a)** evaluates to `"35"`
- **b)** evaluates to `8`
- **c)** raises `TypeError`
- **d)** raises `SyntaxError`

W1-M04 [what] How does Rust manage memory?

- **a)** The programmer calls `free` for every allocation
- **b)** A garbage collector frees unreachable objects at run time
- **c)** Ownership rules, checked by the compiler, guarantee each allocation is freed exactly once
- **d)** It never frees memory until the program ends

W1-M05 [what] Where and why was C created?

- **a)** By Dennis Ritchie at Bell Labs, to write the Unix operating system
- **b)** By Bjarne Stroustrup at Bell Labs, to add classes to Simula
- **c)** By Donald Knuth, to typeset *The Art of Computer Programming*
- **d)** By Guido van Rossum, as a teaching language

W1-M06 [what] An **Abstract Data Type** specifies:

- **a)** how the data is laid out in memory
- **b)** which programming language the structure must be written in
- **c)** the class hierarchy of the implementation
- **d)** the operations, what they mean and what they cost — nothing about storage

W1-M07 [how] Which of these is the *declarative (comprehension)* style from Lecture 01?

- **a)** `for n in numbers: if n % 2 == 0: total += n * n`
- **b)** `total = sum(n * n for n in numbers if n % 2 == 0)`
- **c)** `total = sum(map(lambda n: n * n, filter(lambda n: n % 2 == 0, numbers)))`
- **d)** `while i < len(numbers): ...`

W1-M08 [what] What does the TIOBE index actually count?

- **a)** search-engine results for "X programming"
- **b)** the quality of each language
- **c)** commits on GitHub
- **d)** self-reported usage in a developer survey

W1-M09 [why] In the course's three levels of knowledge, which is **know-why**?

- **a)** You can define a stack
- **b)** You can implement a stack that passes its tests
- **c)** You can say why a stack and not a queue, and what it costs

- **d)** You can recite the Big-O of every stack operation

W1-M10 [what] To pass this course, a student needs:

- **a)** at least 50% overall
- **b)** at least 60% overall and at least 30% of the final exam
- **c)** at least 60% overall, whatever the final exam mark
- **d)** at least 30% of the final exam, whatever the overall mark

W1-M11 [how] What is `-7 // 2`?

- **a)** `-3`
- **b)** `-3.5`
- **c)** `3`
- **d)** `-4`

W1-M12 [how] What is `-7 % 3`?

- **a)** `2`
- **b)** `-1`
- **c)** `1`
- **d)** `-2`

W1-M13 [how] What is `2 ** 3 ** 2`?

- **a)** `64`
- **b)** `512`
- **c)** `36`
- **d)** `12`

W1-M14 [how] What is `type(8 / 4)`?

- **a)** `<class 'int'>`
- **b)** `<class 'str'>`
- **c)** `<class 'float'>`
- **d)** it raises `TypeError`

W1-M15 [how] With `word = "Python"`, what is `word[1:4]`?

- **a)** `"Pyt"`
- **b)** `"ytho"`
- **c)** `"ython"`
- **d)** `"yth"`

W1-M16 [how] What is `"Mansoura"[::-2]`?

- **a)** `"ausa"`
- **b)** `"aruosnaM"`
- **c)** `"Mnor"`
- **d)** `"arsa"`

W1-M17 [how] After `a = [1, 2, 3]`, `b = a`, `b.append(4)`, what is `a`?

- **a)** `[1, 2, 3]`

- **b)** `[4, 1, 2, 3]`
- **c)** `[1, 2, 3, 4]`
- **d)** `[[1, 2, 3], 4]`

W1-M18 [how] After `a = [1, 2]` and `c = a[:]`, what are `c is a` and `c == a`?

- **a)** `True`, `True`
- **b)** `False`, `True`
- **c)** `True`, `False`
- **d)** `False`, `False`

W1-M19 [how] `grid = [[0] * 2] * 2`, then `grid[0][0] = 1`. What is `grid`?

- **a)** `[[1, 0], [0, 0]]`
- **b)** `[[1, 1], [0, 0]]`
- **c)** `[[1, 0], [1, 0]]`
- **d)** `[[1, 1], [1, 1]]`

W1-M20 [what] What is `round(2.5) + round(3.5)`?

- **a)** 6
- **b)** 7
- **c)** 5
- **d)** 8

W1-M21 [why] Why is `0.1 + 0.2 == 0.3` `False` in Python?

- **a)** Python has a bug in float addition
- **b)** `0.1` is stored as an `int`
- **c)** == compares identity, not value, for floats
- **d)** `0.1`, `0.2` and `0.3` cannot be stored exactly in binary floating point

W1-M22 [how] What happens when you run `s = "abc"` then `s[0] = "z"`?

- **a)** `s` becomes `"zbc"`
- **b)** a `TypeError` is raised, because strings are immutable
- **c)** a new string `"zbc"` is created and `s` refers to it
- **d)** an `IndexError` is raised

W1-M23 [how] A function's body is `print(width * height)` and nothing else. What does `x = area(3, 4)` store in `x`?

- **a)** `12`
- **b)** `"12"`
- **c)** `None`
- **d)** nothing — it is a `NameError`

W1-M24 [how] What does `int("4.5")` do?

- **a)** returns 4
- **b)** returns 5
- **c)** returns `4.5`
- **d)** raises `ValueError`

W1-M25 [why] In C, reading `a[10]` from a 5-element array returns whatever happens to be in memory; in Python, `a[10]` on a 5-element list raises `IndexError`. Which explanation matches Lecture 01?

- **a)** C's compilers have a long-standing bug that was never fixed
- **b)** Python checks the index because it is a compiled language
- **c)** C was designed as a portable assembler that trusts the programmer, so it does not pay for a bounds check; Python chose to fail loudly instead
- **d)** C arrays are stored as linked lists, so the index cannot be checked

W1-M26 [how] What does Python print for `2 ** 64 + 1`?

- **a)** `18446744073709551617`
- **b)** `1`, because the value wraps around past 64 bits
- **c)** it raises `OverflowError`
- **d)** `1.8446744073709552e+19`

W1-M27 [why] Why do the course's exercises never call `input()`?

- **a)** `input()` is too slow for large inputs
- **b)** `input()` always returns an `int`, which is not always wanted
- **c)** `input()` was removed in Python 3
- **d)** a function that reads the keyboard cannot be tested automatically; the exercises take parameters and return results instead

W1-M28 [how] What is `f'{7:03d}|{3.14159:.2f}'`?

- **a)** `'7|3.14'`
- **b)** `'007|3.14'`
- **c)** `'700|3.141'`
- **d)** `'007|3.1'`

W1-M29 [how] What does this print?

```
a = [1, 2, 3]
b = a
a = a + [4]
b.append(5)
print(a, b)
```

- **a)** `[1, 2, 3, 4] [1, 2, 3, 5]`
- **b)** `[1, 2, 3, 4, 5] [1, 2, 3, 4, 5]`
- **c)** `[1, 2, 3, 4] [1, 2, 3, 4, 5]`
- **d)** `[1, 2, 3, 5, 4] [1, 2, 3, 5]`

W1-M30 [why] `xs = [42, "hello", 3.14]` holds an `int`, a `str` and a `float` in one list. Why is that possible?

- **a)** Python converts every element to a string when it is stored
- **b)** a list declares a union type when it is created
- **c)** a list grows when a bigger element is added

- **d)** a list stores **references** to objects, not the objects themselves, and every reference has the same size

W1-M31 [what] Lecture 01 lists three properties we care about in an algorithm. In the lecture's order, they are:

- **a)** cost in time, cost in space, correctness
- **b)** correctness, cost in time, cost in space
- **c)** cost in time, cost in space, readability
- **d)** readability, correctness, cost in time

W1-M32 [why] You time linear and binary search on a sorted list of 20 items, and linear search is faster. Does this contradict "binary search is $O(\log n)$, linear search is $O(n)$"?

- **a)** No: Big-O ignores constant factors and describes how the cost grows for large n; at small n, the constants can win
- **b)** Yes: the measurement must have been wrong
- **c)** No: binary search is $O(n)$ when n is below 100
- **d)** Yes: Big-O applies only to unsorted input

W1-M33 [how] An $O(n^2)$ program takes 2 seconds on today's data. The data grows **ten times** larger. About how long will it take now?

- **a)** 20 seconds
- **b)** 40 seconds
- **c)** 200 seconds
- **d)** 4 seconds

W1-M34 [why] TIOBE ranked Visual Basic above Rust in September 2026. What does Lecture 01 conclude from rankings like this?

- **a)** Visual Basic is the better language for new projects
- **b)** The index counts search-engine hits, which says nothing about quality; use rankings only to see trends and to check that a language is not dying, never to choose a language for a task
- **c)** Rust is dying and should not be learnt
- **d)** All rankings are worthless and should never be read

W1-M35 [what] What is `'a,b,,c'.split(',')`?

- **a)** `['a', 'b', '', 'c']`
- **b)** `['a', 'b', 'c']`
- **c)** `['a,b,,c']`
- **d)** `['a', 'b', ',c']`

W1-M36 [why] A shop stores prices in pounds as `float` values and adds them up. Why does Lab 01 say not to, and what should it do instead?

- **a)** floats are slower than ints; it should use strings
- **b)** floats cannot hold negative values; it should use `abs()`
- **c)** floats overflow above a million; it should use smaller units

- **d)** amounts like 0.10 cannot be stored exactly in binary, so totals drift; it should store whole piasters as `int`

W1-M37 [what] What do `'banana'.find('z')` and `'banana'.index('z')` do?

- **a)** both return `-1`
- **b)** `find` returns `-1`; `index` raises `ValueError`
- **c)** `find` raises `ValueError`; `index` returns `-1`
- **d)** `find` returns `None`; `index` raises `IndexError`

W1-M38 [why] Lecture 01 calls Python's built-in `list`, `dict`, `sorted` and `heapq` "the honest case against Python — for us". Why are they a problem in **this** course?

- **a)** They already *are* the structures the course studies, so using them would hide the mechanism and the cost that the course is about
- **b)** They are slower than any structure a student can write
- **c)** They are not part of the Python standard library
- **d)** They make the tests run too fast to be measured

W1-M39 [what] Which pairing of designer, language and decision is correct?

- **a)** Bjarne Stroustrup — Python — readability is worth paying for
- **b)** Guido van Rossum — Go — fewer features, faster builds
- **c)** Bjarne Stroustrup — C++ — leave no room for a lower-level language
- **d)** Graydon Hoare — C — trust the programmer

W1-M40 [why] Lecture 01 says you cannot write badly-indented Python. Why?

- **a)** Python editors re-indent your code before it runs
- **b)** Python ignores indentation, so any indentation is fine
- **c)** the interpreter repairs inconsistent indentation as it reads the file
- **d)** indentation is part of Python's syntax: indent differently and you get a different program (or an error), a decision made for readability

W1-M41 [how] What does this print?

```
def min_max(a, b):
    if a < b:
        return a, b
    return b, a

result = min_max(9, 2)
print(type(result).__name__, result[0])
```

- **a)** `tuple 2`
- **b)** `list 2`
- **c)** `tuple 9`
- **d)** `int 2`

W1-M42 [how] What does this print?

```
grid = [[0, 0], [0, 0]]
copy = grid[:]
copy[0][0] = 9
copy[1] = [7, 7]
print(grid)
```

- **a)** `[[0, 0], [0, 0]]`
- **b)** `[[9, 0], [0, 0]]`
- **c)** `[[9, 0], [7, 7]]`
- **d)** `[[0, 0], [7, 7]]`

Part B — Short answer and essay

W1-E1 [why] *(4 marks)* "All programming languages are the same; only the syntax differs." Explain what is true in this statement and what is wrong with it. Give one concrete example of a design decision that makes two languages genuinely different.

W1-E2 [why] *(4 marks)* Explain the two independent axes of type systems. Place Python and JavaScript on both axes, and show with one expression how their behaviour differs.

W1-E3 [what] *(3 marks)* Describe the three approaches to memory management from Lecture 01, naming one language that uses each. Which one does Python use, and why does that make data structures harder to *see* in Python?

W1-E4 [why] *(4 marks)* Distinguish an **Abstract Data Type** from a **data structure**. Illustrate with the Stack ADT and two different structures that implement it.

W1-E5 [why] *(3 marks)* Give two reasons the course uses Python, and the honest drawback of Python *for this course*. How does the course deal with that drawback?

W1-E6 [why] *(3 marks)* "In Python a variable is not a box holding a value; it is a name bound to an object." Explain this with a short code example and a diagram, and state one consequence for lists.

W1-E7 [why] *(4 marks)* "Measure, do not assert." Explain why the course asks you to measure an algorithm's cost, what a measurement shows that a Big-O table does not, and one way a measurement can mislead you, with how to avoid it.

W1-E8 [why] *(3 marks)* "`print` is not `return`." Explain the difference with a two-line function, say what a test sees in each case, and explain why the exercises use parameters and `return` rather than `input()` and `print()`.

W1-E9 [why] *(4 marks)* "Which language is best?" is not a question. Rewrite it as the real questions it hides, and explain why the **ecosystem** of a language matters more than its syntax, naming at least three parts of an ecosystem.

W1-E10 [how] *(4 marks)* Compare Python's `int` and `float` with C's `int`: what happens when a value gets very large, what that costs, and why `0.1 + 0.2 == 0.3` is `False`. State the two rules

Lab 01 gives for working with floats.

Part C — Trace the code

For each program, write **exactly** what is printed.

W1-T1 [how]

```
print(17 // 5, 17 % 5, -17 // 5, -17 % 5)
print(divmod(23, 4), 2 ** 10, 7 / 2)
```

W1-T2 [how]

```
s = "Data Structures"
print(s[0], s[-1], s[5:9], s[:4].upper())
print(s.split(), len(s), s.count("t"))
```

W1-T3 [how]

```
a = [1, 2, 3]
b = a
c = a[:]
a.append(4)
b[0] = 99
c = c + [5]
print(a, b, c)
a += [6]
print(b)
```

W1-T4 [how]

```
n, count = 1000, 0
while n > 1:
    n = n // 3
    count += 1
print(n, count)
```

W1-T5 [how]

```
name, mark = "Omar", 87.456
print(f"{name:>6}|{mark:.1f}|{mark:8.2f}|{len(name):03d}")
```

W1-T6 [how]

```
a, b = 0, 1
result = []
while len(result) < 8:
    result.append(a)
    a, b = b, a + b
```

```
print(result)
print(result[-3:], result[::3])
```

Part D — Find and fix the bug

Each function has **one** bug. Say what goes wrong, for which input, and fix it.

W1-B1 [how] Should return the initials of a name, e.g. `"H.E."`.

```
def initials(full_name):
    result = ""
    for word in full_name.split(" "):
        result += word[0].upper() + "."
    return result
```

W1-B2 [how] Should return the average of a non-empty list.

```
def average(values):
    total = 0
    i = 0
    while i < len(values):
        total += values[i]
    return total / len(values)
```

W1-B3 [how] Should return a copy of `values` with `item` added at the end, **without changing** `values`.

```
def with_item(values, item):
    result = values
    result.append(item)
    return result
```

Part E — Write the code

Write these in `practice/week01.py` and check with `pytest tests/test_practice_week01.py -v`. The docstrings in that file are the full specification.

W1-C1 [how] `count_vowels(text)` — how many of a, e, i, o, u appear in `text`, in either case. `count_vowels("Data Structures")` is 5.

W1-C2 [how] `second_largest(values)` — the largest value strictly smaller than the maximum, in **one pass** and **without sorting**. Raise `ValueError` if there is no such value. `second_largest([4, 9, 2, 9, 7])` is 7.

W1-C3 [how] `compress(text)` — run-length encoding: `compress("aaabcc")` is `"a3b1c2"`.

W1-C4 [how] `is_anagram(first, second)` — same letters, same counts, ignoring case and spaces. `is_anagram("Dormitory", "dirty room")` is `True`.